МИНОБРНАУКИ РОССИИ САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА

Кафедра вычислительной техники

# ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

По теме: «Управление памятью»

Студент гр. 4312:

Устинов В. А.

Преподаватель:

Тимофеев А. В.

Санкт-Петербург

2026

## Цель работы

Исследовать механизмы управления виртуальной памятью Win32. **Задание**

## 2.1. Исследовать виртуальное адресное пространство процесса

В ходе выполнения первой части лабораторной работы было создано и протестировано консольное приложение, исследующее виртуальную память Windows через WinAPI. Приложение состоит из 7 пунктов: получение системной информации, запрос состояния памяти, выделение/освобождение, запись и изменение защиты страниц. Интерфейс приложения представлен ниже (см Рис. 1).

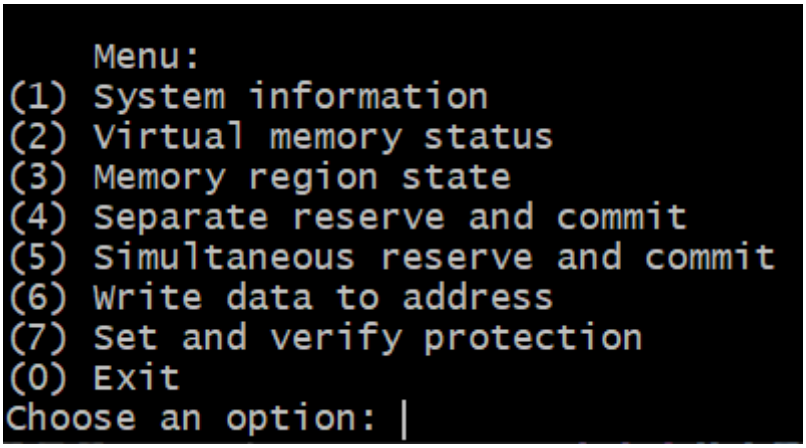


*Рис. 1*

1) Получение информации о вычислительной системе

При вызове первой опции вызывается функция *getSystemInfo*, которая выводит информацию о системе пользователя такую как:

- архитектура процессора;
- размер страницы памяти;
- минимальный и максимальный адрес пользовательского пространства;
- маску активных процессоров и их количество;
- параметры процессора.

Ниже представлена информация о моей системе (см Рис. 2).

```
Choose an option: 1

      System information:
Processor architecture:       9
Page size:                    4096 bytes
Minimum application address:  0000000000010000
Maximum application address:  00007fffffffeffff
Active processor mask:        0xFFFF
Number of processors:         16
Processor type:               8664
Allocation granularity:       65536 bytes
Processor level:              25
Processor revision:           8450
```

*Рис. 2*

Программа показала, что у меня 64-разрядная система с 16 логическими процессорами, размер страницы одной моей страницы памяти равен 4 КБ, гранулярность выделения 64 КБ, а пользовательское адресное пространство процесса размещено от 0x10000 до 0x7FFFFFFFFFFF.

2) Определение статуса виртуальной памяти

При вызове второй опции вызывается функция *GlobalMemoryStatus*, которая показывает текущее состояние физической и виртуальной памяти. Важными атрибутами являются процент загрузки памяти, общий и доступный объем для физической памяти/файла подкачки/виртуального адресного пространства процесса. Ниже представлен вывод программы (см Рис. 3).

```
Choose an option: 2

      Memory status:
Memory load:                    42%
Total physical memory:          4216254464 bytes
Available physical memory:      2657845248 bytes
Total page file:                2068770816 bytes
Available page file:            1780813824 bytes
Total virtual memory:           4294836224 bytes
Available virtual memory:       4244189184 bytes
```

*Рис. 3*

Из вывода приложения видно, что загруженность моей памяти составляет 42%, из установленных в системе 3,93 ГБ ОЗУ осталось 2,48 ГБ. Файл подкачки размером 1,93 ГБ практически не востребован, так как занято всего около 275 МБ, в системе доступно 1,66 ГБ. Общий объём виртуального адресного пространства процесса составляет ровно 4 ГБ, из которых свободно 3,95 ГБ, что указывает на 32-битный характер исполняемой программы.

3) Определение состояния конкретного участка памяти по заданному с клавиатуры адресу

При вызове третьей опции вызывается функция *VirtualQuery*, которая выводит информацию о базовом адресе региона и выделения, размере региона, текущей и первоначальной защите. Ниже представлен вывод программы (см Рис. 4).

```
Choose an option: 3
Enter address (in hexadecimal): 0x0000000000010000

      Memory region state at address 0000000000010000:
Base address:                 0000000000010000
Allocation base:              0000000000000000
Allocation protect:           N/A
Region size:                  2147287040 bytes
State:                        MEM_FREE
Current protect:              PAGE_NOACCESS
Type:                         N/A
```

*Рис. 4*

После ввода адреса, программа запросила состояние памяти по пользовательскому адресу и получила результат, что адрес — это свободный регион размером почти 2 ГБ. Поля Allocation base, Allocation protect и Type отображают N/A поскольку для свободной памяти эти атрибуты не определены. Защита *PAGE_NOACCESS* является формальным признаком отсутствия доступа к незанятым страницам. Состояние *MEM_FREE* означает, что данный диапазон адресов не зарезервирован и не зафиксирован.

4) Раздельное резервирование региона и передачу ему физической памяти в автоматическом режиме и в режиме ввода адреса начала региона

При вызове четвертой опции приложения вызываются функции *VirtualAlloc* и *VirtualFree*, которые реализуют двухшаговое выделение памяти. Сначала *VirtualAlloc* резервирует диапозон адресов без реальной памяти через флаг *MEM_RESERVE*, затем он передаёт физическую память уже зарезервированному региону через флаг *MEM_COMMIT*. В конце регион освобождается с помощью *VirtualFree* через флаг *MEM_RELEASE*. Ниже представлена демонстрация опции (см Рис. 5).

```
Choose an option: 4
Enter region size in bytes: 1048576
Enter starting address (0 - automatic): 0
step 1: Region reserved. Address: 0000018243e30000, size: 1048576 bytes
step 2: Physical memory committed to region. Address: 0000018243e30000
step 3: Region freed.
```

*Рис. 5*

Программа успешно выполнила двухшаговое выделение виртуальной памяти размером 1 МБ. Система сама выбрала свободный диапазон и вернула адрес 0x000001B707780000. На первом шаге *VirtualAlloc* зарезервировал этот непрерывный блок адресов без выделения физической памяти. На втором шаге *VirtualAlloc* передал региону реальную физическую память. Адрес после коммита остался тем же, поскольку память выделялась уже внутри зарезервированного диапазона. В конце регион был полностью освобождён вызовом *VirtualFree* .

5) Одновременное резервирование региона и передача ему физической памяти в автоматическом режиме и в режиме ввода адреса начала региона

При вызове 5 опции приложения происходит вызов аналогичных функций, что и в четвертом пункте отчета. Однако в данном варианте они вызываются не раздельно, а совместно. Ниже представлен вывод программы (см Рис. 6).

```
Choose an option: 5
Enter region size in bytes: 1048576
Enter starting address (0 - automatic): 0
step 1: Region allocated. Address: 00000238b4600000, size: 1048576 bytes
step 2: Region freed (VirtualFree).
```

*Рис. 6*

При одновременном использовании флагов *MEM_RESERVE* и *MEM_COMMIT*, пользователь получает готовый к использованию регион за один вызов функции *VirtualAlloc*, программа автоматиченски подбирает свободный адрес.

6) Запись данных в ячейки памяти по заданным с клавиатуры адресам

При вызове шестой опции приложения пользователь может записать информацию в свободную ячейку памяти. Ниже продемонстрирована работа программы (см Рис. 7).

```
Choose an option: 6
Enter starting address (0 - automatic): 0
Auto-allocated address: 000002c9997f0000
Enter string to write: Hello, World!
Written to address 000002c9997f0000: "Hello, World!"
Memory freed.
```

*Рис. 7*

Программа успешно выполнила запись данных по автоматически выделенному адресу. Был сделан вызов *VirtualAlloc*, и система сама выбрала свободный участок памяти по адресу 0x000002C9997F0000 размером 4 КБ с правами на чтение и запись. Затем в эту область через *memcpy* была скопирована строка "Hello, World!". После успешной записи программа вывела подтверждение, прочитав данные обратно из того же адреса и показав их на экране.

7) Установка защиты доступа для заданного региона

При вызове седьмого пункта пользователь может изменить защиту региона, через функцию VirtualProtect. Демонстрация работы представлена ниже (см Рис. 8).

```
Choose an option: 7
Enter region address (hex, 0 - automatic): 0
Enter region size in bytes: 1048576
Auto-allocated address: 0000029e051d0000, size: 1048576 bytes
     Choose protection type:
(1) PAGE_READONLY
(2) PAGE_READWRITE
(3) PAGE_NOACCESS
(4) PAGE_EXECUTE_READ
Choose: 1
[INFO] Protection changed.
       Old protection: PAGE_READWRITE
       New protection: PAGE_READONLY
[INFO] Verification: current protection = PAGE_READONLY
[INFO] Auto-allocated memory freed.
```

*Рис. 8*

Программа выполнила изменение защиты памяти для автоматически выделенного региона размером 1 МБ. Система сама выделила регион по адресу 0x0000029E051D0000 с начальной защитой PAGE_READWRITE. Затем была выбрана опция 1 — установка защиты PAGE_READONLY. Функция VirtualProtect успешно изменила атрибуты страниц: старая защита PAGE_READWRITE была заменена на PAGE_READONLY. Проверка через VirtualQuery подтвердила, что тип защиты был изменен.

## Выводы по заданию 2.1

В ходе выполнения лабораторной работы были исследованы механизмы управления виртуальной памятью в операционной системе Windows с использованием функций WinAPI. Разработанное консольное приложение позволило на практике изучить структуру виртуального адресного пространства процесса, способы выделения и освобождения памяти, а также методы управления защитой страниц.

С помощью функции *GetSystemInfo* были определены основные характеристики системы: 64-разрядная архитектура процессора, размер страницы 4 КБ, гранулярность выделения 64 КБ, пользовательское адресное пространство от 0x10000 до 0x7FFFFFFFFFFF. Функция *GlobalMemoryStatus* показала текущее состояние памяти: загруженность 42%, доступно 2,48 ГБ физической памяти из 3,93 ГБ, файл подкачки практически не используется, а общий объём виртуального адресного пространства процесса составил 4 ГБ, что подтвердило 32-битный характер исполняемой программы.

Исследование конкретного участка памяти через *VirtualQuery* по минимальному пользовательскому адресу показало, что незанятые регионы имеют состояние *MEM_FREE* с защитой *PAGE_NOACCESS*, а их атрибуты не определены. Размер свободного региона в нижней части адресного пространства составил почти 2 ГБ. Эксперименты с выделением памяти продемонстрировали два подхода: раздельное резервирование и коммит и их одновременное выполнение. Оба способа работают корректно, но раздельный подход даёт более гибкий контроль над использованием физической памяти.

Запись данных в память через *memcpy* требует предварительной проверки состояния и защиты региона, иначе возникает ошибка доступа. Реализованное автоматическое выделение памяти при вводе нулевого адреса сделало операцию записи удобной и безопасной. Функция *VirtualProtect* успешно изменяет защиту страниц: в эксперименте защита была сменена с *PAGE_READWRITE* на *PAGE_READONLY*, что подтвердилось проверкой через *VirtualQuery*. Все изученные механизмы работают стабильно и соответствуют документации WinAPI, обеспечивая надёжное управление виртуальной памятью процесса.

## Задание 2.2 Использование проецируемых файлов для обмена данными между процессами

В ходе выполнения второй части лабораторной работы были созданы и протестированы два консольных приложения — сервер и клиент, они осуществляющие обмен данными через механизм проецируемых в память файлов с использованием WinAPI. Интерфейсы приложений представлены ниже (см Рис. 9, 10).

```
[INFO] Client started. PID = 11800

    Client menu:
(1) Perform mapping
(2) Read data
(3) Exit
Choose an option: |
```

*Рис. 9*

```
[INFO] Server started. PID = 17272

    Server menu:
(1) Perform mapping
(2) Write data
(3) Exit
Choose an option: |
```

*Рис. 10*

1) Сервер (писатель)

Приложение-сервер выполняет три основные операции: создание и проецирование файла в память, запись данных и завершение работы с удалением файла.

1.1) Создание и проецирование файла

При выборе первого пункта меню сервер вызывает функции *CreateFile*, *CreateFileMapping* и *MapViewOfFile*. Сначала создаётся физический файл shared_file.dat на диске, который заполняется нулями до размера 4096 байт. Затем создаётся объект отображения файла с именем Local\SharedFileMap, доступный другим процессам через механизм локального пространства имён. После этого файл проецируется в адресное пространство сервера, и программа получает указатель на начало отображённой области. Ниже представлен вывод программы (см Рис. 11).

```
Choose an option: 1
[INFO] File 'shared_file.dat' created and mapped into memory.
       Mapping name: Local\SharedFileMap
       Mapping address: 0000028358a70000
```

*Рис. 11*

Программа успешно создала файл, объект отображения и спроецировала его в память. Адрес отображения в виртуальном адресном пространстве сервера был выведен на экран.

1.2) Запись данных и ожидание подтверждения

При выборе второго пункта меню сервер запрашивает у пользователя строку, сбрасывает байт-подтверждения (последний байт отображённой области), копирует строку в начало разделяемой памяти и сбрасывает изменения на диск через *FlushViewOfFile*. После этого сервер переходит в режим ожидания: в цикле с интервалом 100 мс он проверяет последний байт области, ожидая, пока клиент не запишет туда значение \1 в качестве подтверждения прочтения. Таймаут ожидания составляет 30 секунд. Ниже продемонстрирована работа записи и подтверждения (см Рис. 12).

```
Choose an option: 2
[Server] Enter string to write: Hello, World!
[INFO] Written: "Hello, World!"
[INFO] Waiting for client...
[INFO] Waiting for client confirmation... (2 s)
[INFO] Waiting for client confirmation... (4 s)
[INFO] Client confirmed reading.
```

*Рис. 12*

Сервер записал строку в разделяемую память и через некоторое время получил подтверждение от клиента, что свидетельствует об успешном двустороннем взаимодействии.

1.3) Завершение работы

При выборе третьего пункта меню сервер отменяет проецирование через *UnmapViewOfFile*, закрывает дескрипторы объекта отображения и файла, а затем удаляет файл shared_file.dat с диска. Ниже продемонстрирован вывод программы (см Рис. 13).

```
Choose an option: 3
[INFO] File 'shared_file.dat' deleted.
[INFO] Server terminated.
```

*Рис. 13*

2) Клиент (читатель)

Приложение-клиент выполняет три операции: подключение к существующему объекту отображения, чтение данных и завершение работы.

2.1) Проецирование файла

При выборе первого пункта меню клиент вызывает *OpenFileMappingA* для открытия уже существующего объекта отображения по имени Local\SharedFileMap, после чего проецирует его в своё адресное пространство через *MapViewOfFile*. Если сервер ещё не создал объект отображения, клиент сообщает об ошибке и предлагает сначала запустить сервер. Ниже продемонстрирован вывод программы (см Рис. 14).

```
Choose an option: 1
[INFO] File mapped into memory.
       Mapping name: Local\SharedFileMap
       Mapping address: 00000260d1de0000
```

*Рис. 14*

2.2) Чтение данных и отправка подтверждения

При выборе второго пункта меню клиент переходит в режим ожидания, проверяет первый байт отображённой области, и пока он равен \0, программа считает, что сервер ещё не записал данные. Ожидание реализовано в цикле с интервалом 100 мс и таймаутом 30 секунд. Как только первый байт становится ненулевым, клиент считывает строку из разделяемой памяти, выводит её на экран и записывает байт \1 в последний байт области в качестве подтверждения. Изменения сбрасываются на диск вызовом *FlushViewOfFile.* Ниже представлен вывод программы (см Рис. 15).

```
Choose an option: 2
[INFO] Waiting for data from server...
[INFO] Received: "Hello, World!"
[INFO] Acknowledgment sent to server.
```

*Рис. 15*

Программа успешно считала строку из пункта 1.2.

2.3) Завершение работы

При выборе третьего пункта меню клиент отменяет проецирование и закрывает дескриптор объекта отображения. Ниже представлен вывод программы (см Рис. 16).

```
Choose an option: 3
[INFO] View unmapped.
[INFO] Client terminated.
```

*Рис. 15*

## Выводы по заданию 2.2

В ходе выполнения задания 2.2 был исследован механизм обмена данными между процессами с использованием проецируемых в память файлов в операционной системе Windows. Были разработаны два консольных приложения  сервер и клиент, взаимодействующие через именованный объект отображения файла.

Сервер создаёт физический файл на диске, заполняет его нулями до заданного размера 4096 байт, после чего создаёт объект отображения с именем Local\SharedFileMap и проецирует файл в своё адресное пространство. Клиент открывает существующий объект отображения по тому же имени и также проецирует его в свою память. Оба процесса получают указатели на одну и ту же область физической памяти, что обеспечивает прямой обмен данными без использования промежуточных буферов или каналов.

Для синхронизации взаимодействия реализован упрощённый механизм на основе двух байтов: первый байт отображённой области служит индикатором наличия данных от сервера, последний байт используется как флаг подтверждения от клиента. Сервер после записи строки ожидает

появления значения \1 в последнем байте, клиент ожидает появления ненулевого значения в первом байте. Оба процесса используют циклический опрос с интервалом 100 мс и таймаутом 30 секунд. Вызов *FlushViewOfFile* после каждой операции записи гарантирует немедленную фиксацию изменений и их видимость другому процессу.

Практическая проверка подтвердила корректность взаимодействия: сервер успешно создал и спроецировал файл, клиент подключился к объекту отображения, записанная сервером строка была прочитана клиентом, а подтверждение доставлено обратно. После завершения работы сервер корректно освободил все ресурсы и удалил файл с диска.

## Текст программы

1) virtualMemory.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

//состояние защиты в строку
static const char *protect_to_str(DWORD protect)
{
    switch (protect){
    case PAGE_NOACCESS:          return "PAGE_NOACCESS";
    case PAGE_READONLY:          return "PAGE_READONLY";
    case PAGE_READWRITE:         return "PAGE_READWRITE";
    case PAGE_WRITECOPY:         return "PAGE_WRITECOPY";
    case PAGE_EXECUTE:           return "PAGE_EXECUTE";
    case PAGE_EXECUTE_READ:      return "PAGE_EXECUTE_READ";
    case PAGE_EXECUTE_READWRITE: return "PAGE_EXECUTE_READWRITE";
    case PAGE_EXECUTE_WRITECOPY: return "PAGE_EXECUTE_WRITECOPY";
    case 0:                      return "N/A";
    default:                     return "UNKNOWN";
    }
}

//тип региона в строку
static const char *type_to_str(DWORD type)
{
    switch (type){
    case MEM_IMAGE:   return "MEM_IMAGE";
    case MEM_MAPPED:  return "MEM_MAPPED";
    case MEM_PRIVATE: return "MEM_PRIVATE";
    case 0:           return "N/A";
```

```
    default:          return "UNKNOWN";
    }
}

//состояние региона в строку
static const char *state_to_str(DWORD state)
{
    switch (state){
    case MEM_COMMIT:  return "MEM_COMMIT";
    case MEM_FREE:    return "MEM_FREE";
    case MEM_RESERVE: return "MEM_RESERVE";
    default:          return "UNKNOWN";
    }
}

static void do_system_info(void)
{
    SYSTEM_INFO si;
    GetSystemInfo(&si);

    printf("\n      System information:\n");
    printf("Processor architecture:       %u\n",   si.wProcessorArchitecture);
    printf("Page size:                    %lu bytes\n", si.dwPageSize);
    printf("Minimum application
address:  %p\n",   si.lpMinimumApplicationAddress);
    printf("Maximum application
address:  %p\n",   si.lpMaximumApplicationAddress);
    printf("Active processor mask:        0x%lX\n", si.dwActiveProcessorMask);
    printf("Number of processors:         %lu\n",   si.dwNumberOfProcessors);
    printf("Processor type:               %lu\n",   si.dwProcessorType);
    printf("Allocation granularity:       %lu bytes\n",
si.dwAllocationGranularity);
    printf("Processor level:              %u\n",   si.wProcessorLevel);
    printf("Processor revision:           %u\n",   si.wProcessorRevision);
}

static void do_memory_status(void)
{
    MEMORYSTATUS ms;
    ms.dwLength=sizeof(ms);
    GlobalMemoryStatus(&ms);

    printf("\n      Memory status:\n");
    printf("Memory load:                  %lu%%\n",  ms.dwMemoryLoad);
    printf("Total physical memory:        %lu bytes\n", ms.dwTotalPhys);
    printf("Available physical memory:    %lu bytes\n", ms.dwAvailPhys);
    printf("Total page file:              %lu bytes\n", ms.dwTotalPageFile);
    printf("Available page file:          %lu bytes\n", ms.dwAvailPageFile);
    printf("Total virtual memory:         %lu bytes\n", ms.dwTotalVirtual);
    printf("Available virtual memory:     %lu bytes\n", ms.dwAvailVirtual);
}
```

```
static void do_virtual_query(void)
{
    char buf[64];
    printf("Enter address (in hexadecimal): ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){
        printf("[ERROR] Input error!\n");
        return;
    }

    LPVOID addr=(LPVOID)(uintptr_t)strtoull(buf, NULL, 16);

    MEMORY_BASIC_INFORMATION mbi;
    SIZE_T result=VirtualQuery(addr, &mbi, sizeof(mbi));
    if(result==0){
        printf("[ERROR] VirtualQuery failed with error: %lu\n", GetLastError());
        return;
    }

    printf("\n      Memory region state at address %p:\n", addr);
    printf("Base address:                 %p\n",   mbi.BaseAddress);
    printf("Allocation base:              %p\n",   mbi.AllocationBase);
    printf("Allocation
protect:           %s\n",   protect_to_str(mbi.AllocationProtect));
    printf("Region size:                  %zu bytes\n", mbi.RegionSize);
    printf("State:                        %s\n",   state_to_str(mbi.State));
    printf("Current protect:              %s\n",   protect_to_str(mbi.Protect));
    printf("Type:                         %s\n",   type_to_str(mbi.Type));
}

static void do_reserve_then_commit(void)
{
    char buf[64];
    SIZE_T size=0;
    LPVOID hint=NULL;

    printf("Enter region size in bytes: ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }
    size=(SIZE_T)strtoull(buf, NULL, 10);
    if(size==0){ printf("[ERROR] Invalid size.\n"); return; }

    printf("Enter starting address (0 - automatic): ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }
    hint=(LPVOID)(uintptr_t)strtoull(buf, NULL, 16);

    //резервирование региона без передачи физической памяти
    LPVOID reserved=VirtualAlloc(hint, size, MEM_RESERVE, PAGE_NOACCESS);
    if(reserved==NULL){
```

```
        printf("[ERROR] VirtualAlloc (MEM_RESERVE) failed with error: %lu\n",
GetLastError());
        return;
    }
    printf("step 1: Region reserved. Address: %p, size: %zu bytes\n", reserved,
size);

    //передача физической памяти зарезервированному региону
    LPVOID committed=VirtualAlloc(reserved, size, MEM_COMMIT, PAGE_READWRITE);
    if(committed==NULL){
        printf("[ERROR] VirtualAlloc (MEM_COMMIT) failed with error: %lu\n",
GetLastError());
        VirtualFree(reserved, 0, MEM_RELEASE);
        return;
    }
    printf("step 2: Physical memory committed to region. Address: %p\n",
committed);

    //освобождение памяти
    VirtualFree(committed, 0, MEM_RELEASE);
    printf("step 3: Region freed.\n");
}

static void do_reserve_and_commit(void)
{
    char buf[64];
    SIZE_T size=0;
    LPVOID hint=NULL;

    printf("Enter region size in bytes: ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }
    size=(SIZE_T)strtoull(buf, NULL, 10);
    if(size==0){ printf("[ERROR] Invalid size.\n"); return; }

    printf("Enter starting address (0 - automatic): ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }
    hint=(LPVOID)(uintptr_t)strtoull(buf, NULL, 16);

    //одновременное резервирование и выделение физической памяти
    LPVOID mem=VirtualAlloc(hint, size, MEM_RESERVE | MEM_COMMIT,
PAGE_READWRITE);
    if(mem==NULL){
        printf("[ERROR] VirtualAlloc (MEM_RESERVE|MEM_COMMIT) failed with error:
%lu\n", GetLastError());
        return;
    }
    printf("step 1: Region allocated. Address: %p, size: %zu bytes\n", mem,
size);
```

```
    //освобождение памяти
    VirtualFree(mem, 0, MEM_RELEASE);
    printf("step 2: Region freed (VirtualFree).\n");
}

static void do_write_memory(void)
{
    char buf[256];
    LPVOID addr=NULL;

    printf("Enter starting address (0 - automatic): ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }
    addr=(LPVOID)(uintptr_t)strtoull(buf, NULL, 16);

    //Если адрес не указан — выделяем память автоматически
    int auto_alloc=0;
    if(addr==NULL){
        addr=VirtualAlloc(NULL, 4096, MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
        if(addr==NULL){
            printf("[ERROR] VirtualAlloc failed with error: %lu\n",
GetLastError());
            return;
        }
        auto_alloc=1;
        printf("Auto-allocated address: %p\n", addr);
    }

    printf("Enter string to write: ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }
    buf[strcspn(buf, "\n")]='\0';

    //Проверка доступности адреса
    MEMORY_BASIC_INFORMATION mbi;
    if(VirtualQuery(addr, &mbi, sizeof(mbi))==0){
        printf("[ERROR] VirtualQuery failed with error: %lu\n", GetLastError());
        if(auto_alloc) VirtualFree(addr, 0, MEM_RELEASE);
        return;
    }
    if(mbi.State != MEM_COMMIT){
        printf("[ERROR] Address %p is not committed (state: %s). Write is
impossible.\n",
               addr, state_to_str(mbi.State));
        if(auto_alloc) VirtualFree(addr, 0, MEM_RELEASE);
        return;
    }
    if(mbi.Protect==PAGE_NOACCESS || mbi.Protect==PAGE_READONLY){
        printf("[ERROR] Address %p is write-protected (%s).\n", addr,
protect_to_str(mbi.Protect));
        if(auto_alloc) VirtualFree(addr, 0, MEM_RELEASE);
```

```
        return;
    }

    //Запись данных
    memcpy(addr, buf, strlen(buf) + 1);
    printf("Written to address %p: \"%s\"\n", addr, (char *)addr);

    //Освобождение, если память была выделена автоматически
    if(auto_alloc){
        VirtualFree(addr, 0, MEM_RELEASE);
        printf("Memory freed.\n");
    }
}

static void do_virtual_protect(void)
{
    char buf[64];
    LPVOID addr=NULL;

    printf("Enter region address (hex, 0 - automatic): ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }
    addr=(LPVOID)(uintptr_t)strtoull(buf, NULL, 16);

    //Автоматическое выделение памяти, если адрес 0
    int auto_alloc=0;
    SIZE_T size=0;

    if(addr==NULL){
        printf("Enter region size in bytes: ");
        if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input
error!\n"); return; }
        size=(SIZE_T)strtoull(buf, NULL, 10);
        if(size==0){ printf("[ERROR] Invalid size.\n"); return; }

        addr=VirtualAlloc(NULL, size, MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
        if(addr==NULL){
            printf("[ERROR] VirtualAlloc failed with error: %lu\n",
GetLastError());
            return;
        }
        auto_alloc=1;
        printf("Auto-allocated address: %p, size: %zu bytes\n", addr, size);
    } else{
        printf("Enter region size in bytes: ");
        if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input
error!\n"); return; }
        size=(SIZE_T)strtoull(buf, NULL, 10);
        if(size==0){ printf("[ERROR] Invalid size.\n"); return; }
    }
```

```
    printf("     Choose protection type:\n");
    printf("(1) PAGE_READONLY\n");
    printf("(2) PAGE_READWRITE\n");
    printf("(3) PAGE_NOACCESS\n");
    printf("(4) PAGE_EXECUTE_READ\n");
    printf("Choose: ");
    if(fgets(buf, sizeof(buf), stdin)==NULL){ printf("[ERROR] Input error!\n");
return; }

    DWORD newProtect;
    switch (atoi(buf)){
    case 1: newProtect=PAGE_READONLY;       break;
    case 2: newProtect=PAGE_READWRITE;      break;
    case 3: newProtect=PAGE_NOACCESS;       break;
    case 4: newProtect=PAGE_EXECUTE_READ;   break;
    default:
        printf("[ERROR] Invalid choice.\n");
        if(auto_alloc) VirtualFree(addr, 0, MEM_RELEASE);
        return;
    }

    DWORD oldProtect=0;
    if(!VirtualProtect(addr, size, newProtect, &oldProtect)){
        printf("[ERROR] VirtualProtect failed with error: %lu\n",
GetLastError());
        if(auto_alloc) VirtualFree(addr, 0, MEM_RELEASE);
        return;
    }
    printf("[INFO] Protection changed.\n");
    printf("       Old protection: %s\n", protect_to_str(oldProtect));
    printf("       New protection: %s\n", protect_to_str(newProtect));

    //Проверка через VirtualQuery
    MEMORY_BASIC_INFORMATION mbi;
    if(VirtualQuery(addr, &mbi, sizeof(mbi)) != 0)
        printf("[INFO] Verification: current protection=%s\n",
               protect_to_str(mbi.Protect));

    if(auto_alloc){
        VirtualFree(addr, 0, MEM_RELEASE);
        printf("[INFO] Auto-allocated memory freed.\n");
    }
}

static void print_menu(void)
{
    printf("\n    Menu:\n");
    printf("(1) System information\n");
    printf("(2) Virtual memory status\n");
    printf("(3) Memory region state\n");
    printf("(4) Separate reserve and commit\n");
```

```
    printf("(5) Simultaneous reserve and commit\n");
    printf("(6) Write data to address\n");
    printf("(7) Set and verify protection \n");
    printf("(0) Exit\n");
    printf("Choose an option: ");
}

int main(void)
{
    char line[64];
    int  choice;

    printf("[INFO] Program started. PID=%lu\n", GetCurrentProcessId());

    for (;;){
        print_menu();
        if(fgets(line, sizeof(line), stdin)==NULL)
            break;
        choice=atoi(line);

        switch (choice){
        case 1: do_system_info();         break;
        case 2: do_memory_status();       break;
        case 3: do_virtual_query();       break;
        case 4: do_reserve_then_commit(); break;
        case 5: do_reserve_and_commit();  break;
        case 6: do_write_memory();        break;
        case 7: do_virtual_protect();     break;
        case 0:
            printf("Program terminated.\n");
            return 0;
        default:
            printf("[ERROR] Invalid menu option!\n");
        }
    }

    return 0;
}
```

2) client.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAPNAME     "Local\\SharedFileMap"
#define FILESIZE    4096            //размер файла в байтах
#define ACK_OFFSET (FILESIZE - 1) //смещение байта-подтверждения

static HANDLE hMapping  = NULL; //дескриптор отображения
static char  *ptr       = NULL; //указатель на отображённую память
```

```c
static void do_map(void)
{
    if(ptr ! =  NULL){
        printf("[ERROR] Файл уже спроецирован.\n");
        return;
    }

    //открытие существующего объекта отображения по имени
    hMapping = OpenFileMappingA(FILE_MAP_ALL_ACCESS, FALSE, MAPNAME);
    if(hMapping == NULL){
        printf("[ERROR] OpenFileMapping завершился с ошибкой: %lu\n",
GetLastError());
        printf("[ERROR] Убедитесь, что сервер выполнил проецирование.\n");
        return;
    }

    //проецирование файла в адресное пространство процесса
    ptr  = (char *)MapViewOfFile(hMapping, FILE_MAP_ALL_ACCESS, 0, 0, FILESIZE);
    if(ptr == NULL){
        printf("[ERROR] MapViewOfFile завершился с ошибкой: %lu\n",
GetLastError());
        CloseHandle(hMapping);
        hMapping = NULL;
        return;
    }

    printf("[INFO] Файл спроецирован в память.\n");
    printf("       Имя отображения: %s\n", MAPNAME);
    printf("       Адрес отображения: %p\n",(void *)ptr);
}

static void do_read(void)
{
    if(ptr == NULL){
        printf("[ERROR] Сначала выполните проецирование(пункт 1).\n");
        return;
    }

    //проверка первого байта(если равен '\0', то сервер ничего не записал)
    printf("[INFO] Ожидание данных от сервера...\n");

    int waited = 0;
    while(ptr[0] == '\0'){
        Sleep(100); //100 мс

        waited++;
        if(waited % 20 == 0)
            printf("[INFO] Время ожидания: %d с\n", waited / 10);
        if(waited > 300){   //30 секунд
```

```
            printf("[INFO] Таймаут ожидания сервера.\n");
            return;
        }
    }

    //чтение строки из общей памяти(без последнего байта — там флаг)
    char buf[FILESIZE];
    int len = ACK_OFFSET <(int)sizeof(buf)? ACK_OFFSET :(int)sizeof(buf)- 1;
    strncpy(buf, ptr, len);
    buf[len] = '\0';

    printf("[INFO] Получено: \"%s\"\n", buf);

    //отправка подтверждения серверу, записываем '\1' в последний байт
    ptr[ACK_OFFSET] = '\1';
    if(!FlushViewOfFile(ptr, FILESIZE))
        printf("[ERROR] FlushViewOfFile: %lu\n", GetLastError());

    printf("[INFO] Подтверждение отправлено серверу.\n");
}

static void do_exit(void)
{
    if(ptr ! =  NULL){
        //отмена проецирования
        UnmapViewOfFile(ptr);
        ptr = NULL;
        printf("[INFO] Проецирование отменено.\n");
    }
    if(hMapping ! =  NULL){
        CloseHandle(hMapping);
        hMapping = NULL;
    }

    printf("[INFO] Работа завершена.\n");
    exit(EXIT_SUCCESS);
}

static void print_menu(void)
{
    printf("\n    Меню клиента:\n");
    printf("(1)Выполнить проецирование\n");
    printf("(2)Прочитать данные\n");
    printf("(3)Завершить работу\n");
    printf("Выберите пункт: ");
}

int main(void)
{
    char line[64];
    int  choice;
```

```
    printf("[INFO] Клиент запущен. PID = %lu\n", GetCurrentProcessId());

    for(;;){
        print_menu();
        if(fgets(line, sizeof(line), stdin) ==  NULL)
            break;
        choice = atoi(line);

        switch(choice){
        case 1: do_map();   break;
        case 2: do_read();  break;
        case 3: do_exit();  break;
        default:
            printf("[ERROR] Неверный пункт меню!\n");
        }
    }

    do_exit();
    return 0;
}
```

3) server.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define FILENAME    "shared_file.dat"
#define MAPNAME     "Local\\SharedFileMap"
#define FILESIZE    4096            //file size in bytes
#define ACK_OFFSET (FILESIZE - 1) //acknowledgment byte offset

static HANDLE hFile    = INVALID_HANDLE_VALUE; //file handle
static HANDLE hMapping = NULL;                 //mapping handle
static char  *ptr      = NULL;                 //pointer to mapped memory


static void do_map(void)
{
    if(ptr != NULL){
        printf("[ERROR] File is already mapped.\n");
        return;
    }

    //create file on disk
    hFile = CreateFileA(
        FILENAME,
        GENERIC_READ | GENERIC_WRITE,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
```

```
        NULL,
        CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
    if(hFile == INVALID_HANDLE_VALUE){
        printf("[ERROR] CreateFile failed with error: %lu\n", GetLastError());
        return;
    }

    //fill file with zeros up to required size
    char buf[FILESIZE];
    memset(buf, 0, sizeof(buf));
    DWORD written;
    if(!WriteFile(hFile, buf, FILESIZE, &written, NULL)|| written != FILESIZE){
        printf("[ERROR] WriteFile failed with error: %lu\n", GetLastError());
        CloseHandle(hFile);
        hFile = INVALID_HANDLE_VALUE;
        return;
    }

    //create file mapping object
    hMapping = CreateFileMappingA(
        hFile,
        NULL,
        PAGE_READWRITE,
        0,
        FILESIZE,
        MAPNAME
    );
    if(hMapping == NULL){
        printf("[ERROR] CreateFileMapping failed with error: %lu\n",
GetLastError());
        CloseHandle(hFile);
        hFile = INVALID_HANDLE_VALUE;
        return;
    }

    //map file into process address space
    ptr =(char *)MapViewOfFile(hMapping, FILE_MAP_ALL_ACCESS, 0, 0, FILESIZE);
    if(ptr == NULL){
        printf("[ERROR] MapViewOfFile failed with error: %lu\n", GetLastError());
        CloseHandle(hMapping);
        CloseHandle(hFile);
        hMapping = NULL;
        hFile    = INVALID_HANDLE_VALUE;
        return;
    }

    printf("[INFO] File '%s' created and mapped into memory.\n", FILENAME);
    printf("       Mapping name: %s\n", MAPNAME);
```

```
    printf("       Mapping address: %p\n",(void *)ptr);
}

static void do_write(void)
{
    if(ptr == NULL){
        printf("[ERROR] Perform mapping first!\n");
        return;
    }

    char message[FILESIZE];
    printf("[Server] Enter string to write: ");
    if(fgets(message, sizeof(message), stdin)== NULL){
        printf("[ERROR] Input error!\n");
        return;
    }
    //remove newline character
    message[strcspn(message, "\n")] = '\0';

    //reset acknowledgment byte before writing
    ptr[ACK_OFFSET] = '\0';

    //write string to the beginning of mapped memory
    strncpy(ptr, message, FILESIZE - 2);
    ptr[FILESIZE - 2] = '\0';

    //flush changes to disk
    if(!FlushViewOfFile(ptr, FILESIZE))
        printf("[ERROR] FlushViewOfFile: %lu\n", GetLastError());

    printf("[INFO] Written: \"%s\"\n", ptr);
    printf("[INFO] Waiting for client...\n");

    //wait for client acknowledgment('\1' at the end of buffer)
    int waited = 0;
    while(ptr[ACK_OFFSET] != '\1'){
        Sleep(100); //100 ms

        waited++;
        if(waited % 20 == 0)
            printf("[INFO] Waiting for client confirmation...(%d s)\n", waited /
10);
        if(waited > 300){   //30 seconds
            printf("[INFO] Client wait timeout.\n");
            return;
        }
    }
    printf("[INFO] Client confirmed reading.\n");
}

static void do_exit(void)
```

```c
{
    if(ptr != NULL){
        //unmap view
        UnmapViewOfFile(ptr);
        ptr = NULL;
        printf("[INFO] View unmapped.\n");
    }
    if(hMapping != NULL){
        CloseHandle(hMapping);
        hMapping = NULL;
    }
    if(hFile != INVALID_HANDLE_VALUE){
        CloseHandle(hFile);
        hFile = INVALID_HANDLE_VALUE;
    }

    //delete file from disk
    if(DeleteFileA(FILENAME))
        printf("[INFO] File '%s' deleted.\n", FILENAME);
    else if(GetLastError()!= ERROR_FILE_NOT_FOUND)
        printf("[ERROR] DeleteFile failed with error: %lu\n", GetLastError());

    printf("[INFO] Server terminated.\n");
    exit(EXIT_SUCCESS);
}

static void print_menu(void)
{
    printf("\n    Server menu:\n");
    printf("(1)Perform mapping\n");
    printf("(2)Write data\n");
    printf("(3)Exit\n");
    printf("Choose an option: ");
}

int main(void)
{
    char line[64];
    int  choice;

    printf("[INFO] Server started. PID = %lu\n", GetCurrentProcessId());

    for(;;){
        print_menu();
        if(fgets(line, sizeof(line), stdin)== NULL)
            break;
        choice = atoi(line);

        switch(choice){
        case 1: do_map();   break;
        case 2: do_write(); break;
```

```
        case 3: do_exit();  break;
        default:
            printf("[ERROR] Invalid menu option!\n");
        }
    }

    do_exit();
    return 0;
}
```